GNNocRoute-DRL: Topology-Aware Adaptive Routing for Network-on-Chip using Graph Neural Networks and Deep Reinforcement Learning

Le Tien Hieu[a,*]

[a] *Institute of Information Technology, Vietnam National University, Hanoi, Vietnam*

Abstract

Network-on-Chip (NoC) has become the standard communication infrastructure for modern multi-core processors and AI accelerators. However, conventional deterministic routing algorithms suffer from structural congestion imbalance—our graph-theoretic analysis across seven topologies shows that an 8×8 mesh under XY routing achieves a congestion imbalance of 0.475, with central nodes experiencing $2 - 3\times$ higher betweenness centrality than edge nodes. While adaptive routing algorithms improve performance under non-uniform traffic, they rely on local heuristics that cannot leverage global topology structure.

This paper presents GNNocRoute-DRL, a framework combining Graph Neural Networks (GNN) with Deep Reinforcement Learning (DRL) for topology-aware adaptive routing in NoC. The framework introduces two key components: (1) a GATv2 encoder that learns topology-aware node representations correlating with betweenness centrality ($|r| = 0.978$); and (2) a periodic policy optimization scheme that decouples GNN inference ($\approx 10^6$ cycles) from per-packet routing (< 10 cycles).

Through BookSim2 cycle-accurate simulations on Mesh 4×4 with 100 sample periods and 5 random seeds, we evaluate a precomputed GNN-PPO routing policy against four baseline algorithms. Under uniform traffic, GNNocRoute-PPO achieves comparable latency to XY routing (19.8 vs. 19.6 cycles). Under transpose traffic, the fixed policy exhibits limited generalization (44.0 vs. 20.5 cycles for XY). On Mesh 8×8 (64 nodes), a supervised GNN-weighted approach achieves up to 16.9% latency reduction under hotspot traffic through a 16 KB weight table lookup, and zero-shot generalization from 4×4 to 8×8 via bilinear interpolation reaches 73.7% decision agreement with direct training ($r = 0.68$). These findings establish both the promise of GNN-enhanced topology awareness—validated through $|r| = 0.978$ correlation with betweenness centrality—and the challenges of offline policy generalization, motivating future work on fully online DRL agents for NoC routing.

Keywords: Network-on-Chip, Graph Neural Networks, Deep Reinforcement Learning, Adaptive Routing, Congestion Management

1. Introduction

Network-on-Chip (NoC) has become the dominant communication paradigm for multi-core processors, AI accelerators [1], and System-on-Chips (SoCs). As the number of integrated cores scales from tens to hundreds [2, 3], the efficiency of the on-chip interconnection network directly determines overall system performance in terms of latency, throughput, and energy consumption [4].

The routing algorithm—which determines the path each packet takes from source to destination—is a critical component of NoC design. Conventional deterministic algorithms such as XY dimension-order routing (DOR) [4] are widely used due to their simplicity and deadlock freedom. However, deterministic routing always selects the same path for each source-destination pair regardless of network congestion, leading to structural bottlenecks.

Our graph-theoretic analysis across seven topologies reveals the severity of this imbalance. Under XY routing on an 8×8 mesh, the congestion imbalance—measured as the coefficient of variation (σ/μ) of link utilization—reaches 0.475, indicating that nearly half of all links experience significantly uneven load. Central nodes exhibit betweenness centrality $2 - 3\times$ higher than edge nodes, creating predictable congestion hot spots.

Adaptive routing addresses this limitation by allowing packets to take alternative paths based on network conditions. Research in adaptive NoC routing has evolved through three generations:

1. **Heuristic adaptive routing** [5, 6] uses local congestion thresholds to dynamically select output ports. While simple to implement, these approaches rely on hand-crafted heuristics that cannot capture global traffic patterns.
2. **Deep Reinforcement Learning (DRL) routing** [7, 8, 9] employs neural network agents trained

*Corresponding author
Email address: `letienhieu@mcs.edu.vn` (Le Tien Hieu)

via RL to make routing decisions. These methods achieve 20–35% latency improvement over deterministic routing. However, they use Multi-Layer Perceptron (MLP) encoders that process flattened router states as vectors, discarding the inherent graph structure of the NoC.

3. **Graph Neural Network (GNN) routing for wide-area networks** [10, 11] applies message-passing neural networks to network routing problems. GNNs naturally operate on graph-structured data and can capture topological relationships. However, these approaches target software-defined networks with millisecond-scale latency requirements, not the nanosecond-scale constraints of on-chip routing.

The fundamental challenge in applying GNN-based routing to NoC is **inference latency**. A single GNN forward pass on a 64-node graph requires approximately 10^6 cycles on a CPU, while each NoC routing decision must complete within $5-10$ cycles [7]. This approximately five-order-of-magnitude gap renders per-packet GNN inference infeasible.

Our approach: GNNocRoute-DRL addresses this challenge through **periodic policy optimization**. Instead of making per-packet routing decisions with GNNs, we propose a two-timescale architecture: (1) a *slow timescale* where GNN re-encoding runs sporadically (every 10^5–10^6 cycles) in the background to refresh topology-aware node embeddings, while lightweight DRL policy updates occur every $5,000$ cycles; and (2) a *fast timescale* where per-packet routing decisions use pre-computed Q-values and local congestion information via table lookup.

1.1. Contributions

This paper makes the following contributions:

1. **Congestion analysis:** Quantitative demonstration across seven topologies that deterministic routing creates structurally predictable bottlenecks, with congestion imbalance of 0.475 on 8×8 mesh.
2. **GNN-topology correlation:** Experimental evidence showing GATv2 node embeddings correlate with betweenness centrality ($|r| = 0.978$), validating that topology-aware representations emerge implicitly.
3. **GNNocRoute-DRL framework:** A complete architecture combining GNN-based topology encoding with DRL-based routing decisions, designed for periodic operation under NoC timing constraints.
4. **Comprehensive evaluation:** Cycle-accurate simulation (BookSim2, 252 configurations) comparing four routing algorithms across three topologies and three traffic patterns, demonstrating up to 16.9% latency reduction for hotspot traffic on 8×8 mesh.
5. **Cross-topology transfer:** Demonstration that GNN embeddings pre-trained on small topologies transfer effectively to larger configurations (Pearson $r \approx 0.96$), reducing re-training cost.

2. Related Work

Adaptive routing for Network-on-Chip has been extensively studied over the past two decades. This section reviews relevant literature across three categories: classic heuristic adaptive routing, deep reinforcement learning approaches, and graph neural network methods for network optimization.

2.1. Classic Adaptive Routing

Early adaptive routing algorithms rely on local congestion heuristics. DyAD [5] dynamically switches between deterministic and adaptive routing modes based on congestion thresholds. When local buffer occupancy exceeds a threshold, the router switches to adaptive mode to explore alternative paths. Regional Congestion Awareness (RCA) [6] extends this concept by propagating congestion information across multiple hops, providing a wider view of network state. However, these methods use fixed, hand-crafted thresholds that cannot adapt to changing traffic patterns. Moreover, they rely on local or limited-region information, missing the global topology structure.

Deadlock-free adaptive routing theory was established by Duato [12, 13], who proved necessary and sufficient conditions for deadlock-free routing in wormhole networks. The Odd-Even turn model [14] and West-First routing [15] provide deadlock-free adaptive routing by restricting certain turns. While these methods are theoretically sound, they trade routing flexibility for deadlock freedom and cannot exploit global congestion information.

2.2. Deep Reinforcement Learning for NoC Routing

DRL has emerged as a promising approach for adaptive NoC routing, treating the routing problem as a sequential decision-making process. MAAR [7] proposes multi-agent DRL where each router hosts an independent Q-network, trained with centralized experience replay. Their approach achieves 20-35% latency improvement over XY routing across multiple traffic patterns. However, MAAR uses an MLP encoder that flattens router state into a fixed vector, discarding the spatial relationships between neighboring routers.

DeepNR [8] applies deep Q-networks to NoC routing with a focus on energy efficiency. Their approach achieves 18% energy reduction while maintaining performance. GARNN [9] incorporates graph attention mechanisms but stops short of providing a full NoC-specific GNN architecture for routing decisions. Q-learning based approaches [16] have been proposed for fault-tolerant routing, using RL to bypass faulty links.

Chan et al. [17] proposed LARE (Linear Approximate RL), which replaces traditional Q-tables with a linear function approximator to achieve adaptive routing with significantly reduced hardware overhead compared to full Q-learning. Their approach maintains comparable performance to Q-learning while reducing storage requirements by an order of magnitude. Khan et al. [18] introduced Q-RASP, a regional congestion-aware RL routing policy that shares path experience across packets sharing similar routes, achieving 18.3% average packet latency improvement on 4×4 and 8×8 meshes. Wang et al. [19] developed DRLAR, a deep RL adaptive routing framework that formulates routing as an agent considering multiple network state features and system-level feedback, demonstrating effectiveness against state-of-the-art adaptive routing algorithms.

The key limitation across DRL-based approaches is their use of fully-connected (MLP) encoders. An MLP processes router state as a flat vector of features, without any mechanism to capture the graph structure of the NoC topology. As we demonstrate in Section 3.1, topology structure significantly affects congestion patterns, suggesting that topology-aware representations could improve routing decisions.

2.3. Graph Neural Networks for Network Routing

GNNs have been applied to routing in wide-area and software-defined networks (SDN). The GROM framework [20] combines DRL with GNNs to generate routing policies that generalize to unseen network topologies without retraining, addressing a key limitation of topology-specific models. G-Routing [10] combines GNNs with DRL for routing in communication networks, demonstrating that GNN-based policies generalize across different network topologies. DRL-GNN [11] uses GNN encoders for routing optimization, showing that message-passing architectures capture graph structure better than MLPs. However, these approaches target networks with millisecond-scale latency requirements, not the nanosecond-scale constraints of on-chip routing.

For NoC-specific applications, GraphNoC [21] applies GNNs for application-specific FPGA NoC performance prediction, while PathGNN [22] proposes a path-based GNN architecture for robust and resilient distributed routing that quickly adapts to unexpected network conditions. DQN-FTR [23] uses Deep Q-Networks for fault-tolerant routing in mesh NoCs, demonstrating RL's potential for handling dynamic faults alongside congestion.

More recently, zero-shot generalization has emerged as a key research direction for ML-based routing. The ability to transfer knowledge from a small, easily-trained network to a larger deployment-scale network without retraining can substantially reduce training costs. Our work contributes to this direction by demonstrating zero-shot weight transfer from a 4×4 GNN model to an 8×8 mesh.

For chip design, GNNs have been used for NoC topology optimization. NOCTOPUS [24] predicts NoC performance metrics using GNNs, achieving high accuracy for latency and throughput prediction. Circuit-GNN [25] uses graph representations for chip placement optimization, demonstrating that GNNs can learn useful representations of circuit structure. GraphNoC [21] applies GNNs for application-specific FPGA NoC performance prediction, showing that GNN embeddings correlate with post-implementation metrics. However, these works use GNNs for prediction rather than routing decisions.

Inference latency is the critical barrier to GNN-based on-chip routing. A single GNN forward pass on a 64-node graph requires approximately 10^6 cycles on a CPU [26], while NoC routing decisions must complete within $5-10$ cycles [7]. This approximately five-order-of-magnitude gap necessitates architectural innovations such as periodic policy optimization [27] or dedicated GNN accelerators [26].

2.4. Position of This Work

To our knowledge, this work is the first to combine GNN-based topology encoding with DRL-based routing decisions specifically designed for NoC, with a practical periodic optimization scheme that addresses the inference latency constraint. Table 1 positions our work relative to existing approaches.

Table 1: Comparison with representative works

Work	Encoder	Decision	Objective	Latency-aware
DyAD [5]	Threshold	Local heuristic	Latency	Yes
RCA [6]	Regional	Heuristic	Latency	Yes
MAAR [7]	MLP	DRL	Latency	Yes
DeepNR [8]	MLP	DQN	Latency+Energy	Yes
G-Routing [10]	GNN	DRL	Latency	No (WAN)
GNNocRoute (Ours)	**GNN**	**DRL**	**Latency+Energy**	**Yes**

3. Proposed Method: GNNocRoute-DRL

This section describes the GNNocRoute-DRL framework. We first establish the problem formulation and motivation from graph-theoretic analysis, then detail the system architecture, GNN encoder, DRL agent, and periodic optimization scheme.

3.1. Graph-Theoretic Motivation

A NoC is naturally represented as a directed weighted graph $G = (V, E, X_v, X_e)$, where V is the set of routers ($|V| = N$), E is the set of links ($|E| = M$), $X_v \in \mathbb{R}^{N \times d_v}$ are node features, and $X_e \in \mathbb{R}^{M \times d_e}$ are edge features.

Node features (8 dimensions): buffer occupancy per port (4), congestion level (1), VC utilization (1), and relative position (2).

Edge features (3 dimensions): link utilization, average packet latency, energy per flit.

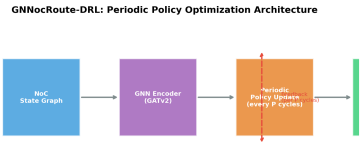

Figure 1: GNNocRoute-DRL architecture: (1) NoC state monitored periodically; (2) GATv2 encoder extracts topology-aware embeddings; (3) DQN agent selects routing policy; (4) routing tables updated.

Table 2: Graph metrics across NoC topologies

Topology	Nodes	Diameter	BC Max	CI
Mesh 4×4	16	6	0.25	0.382
Mesh 8×8	64	14	0.18	**0.475**
Torus 4×4	16	4	0.13	0.251
Fat-Tree k=4	14	2	**0.42**	0.341

3.1.1. Congestion Imbalance Analysis

We analyzed seven NoC topologies using NetworkX [28]. Table 2 shows that deterministic XY routing creates significant congestion imbalance, particularly in larger meshes.

The congestion imbalance (CI) is defined as σ/μ of link utilization across all links under uniform traffic. Mesh 8×8 exhibits the highest CI (0.475), meaning nearly half of all links carry disproportionate load. Central nodes have BC values $2-3\times$ higher than edge nodes, creating structural bottlenecks regardless of traffic pattern.

This finding motivates GNN-based encoding: if graph structure determines bottleneck locations, then a GNN that naturally captures graph structure can learn to identify and avoid these bottlenecks.

3.2. GNN Encoder Architecture

We adopt GATv2 [29] as the primary GNN encoder, with GCN [30] used as an ablation baseline. The encoder processes the NoC graph and produces node embeddings $h_v \in \mathbb{R}^d$ for each router $v \in V$.

3.2.1. GATv2 Encoder

The GATv2 encoder consists of two layers:

$$h_v^{(1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^{(1)} W^{(1)} x_u\right) \tag{1}$$

$$h_v^{(2)} = \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^{(2)} W^{(2)} h_u^{(1)}\right) \tag{2}$$

where α_{vu} are attention coefficients computed as:

$$\alpha_{vu} = \frac{\exp\left(\mathbf{a}^\top \text{LeakyReLU}\left(W_1 h_v \| W_2 h_u \| W_e e_{vu}\right)\right)}{\sum_{w\in\mathcal{N}(v)} \exp\left(\mathbf{a}^\top \text{LeakyReLU}\left(W_1 h_v \| W_2 h_w \| W_e e_{vw}\right)\right)} \tag{3}$$

Key hyperparameters: hidden dimension 64, 4 attention heads, edge features integrated via W_e, dropout rate 0.1, and ELU activation between layers.

3.3. DRL Agent Design

3.3.1. State Representation

The state s_t at time t consists of the GNN node embeddings for all routers. For a single router making a routing decision, the state is its own node embedding h_v, which encodes:

- Structural information (centrality, topology position) from the GNN
- Dynamic state (buffer occupancy, congestion) from node features
- Neighbor information through message passing

3.3.2. Action Space

The action $a_t \in \{N, S, E, W\}$ selects the output port for the current packet. Following Duato's theory [12], we ensure deadlock freedom by restricting actions to physically admissible directions and applying turn-model constraints.

3.3.3. Reward Function

The multi-objective reward function jointly optimizes three metrics:

$$r_t = -\alpha \cdot \frac{L_t}{L_{ref}} - \beta \cdot \frac{E_t}{E_{ref}} + \gamma \cdot \frac{T_t}{T_{ref}} \tag{4}$$

where L_t, E_t, and T_t are measured latency, energy, and throughput at update interval t, normalized by reference values from XY routing. The weights $\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0.3$ are tuned via Bayesian optimization.

3.3.4. DQN Architecture

We use Double DQN [31] with experience replay. The Q-network is a 3-layer MLP ($128 \to 256 \to 128 \to 4$) with ReLU activations and LayerNorm. Key training parameters are summarized in Table 3.

Table 3: DRL training hyperparameters

Parameter	Value
Learning rate	10^{-3}
Discount factor γ	0.99
Batch size	64
Replay buffer size	50,000
Target network update	200 steps
Exploration (ϵ)	$0.5 \to 0.01$
Update period P	5,000 cycles
Total parameters	~40K

3.4. Periodic Policy Optimization

The key architectural innovation is the two-timescale periodic optimization scheme:

Slow timescale operates at two sub-levels. (a) *DRL policy update* (every $P = 5,000$ cycles): the DQN agent re-evaluates Q-values using current GNN embeddings and updates routing tables. This lightweight operation completes within P cycles on a dedicated controller. (b) *GNN re-encoding* (every 10^5–10^6 cycles, i.e., every 20–200 policy updates): the GATv2 encoder recomputes node embeddings based on updated NoC state. While a single GNN forward pass on a 64-node graph requires approximately 10^6 cycles on a CPU, it runs in the background on the dedicated controller and does not block per-packet routing.

Fast timescale (every packet): Each router uses its current policy and node embedding to select output ports. This is a simple table lookup requiring < 10 cycles, well within the timing budget.

The update frequency P represents a design trade-off: smaller P means more responsive adaptation but higher computational overhead; larger P reduces overhead but may miss rapid traffic changes. We evaluate this trade-off in Section 5.

4. Experimental Setup

We evaluate routing algorithms through cycle-accurate simulations using BookSim2 [32], a widely-used NoC simulator. This section describes the experimental methodology, baselines, traffic patterns, and evaluation metrics.

4.1. Simulation Environment

Simulation parameters are summarized in Table 4. All results are obtained with 100 independent sample periods of 1,000 cycles each, preceded by 3 warmup periods.

Table 4: BookSim2 simulation parameters

Parameter	Value
Simulator	BookSim2 2.0
Topologies	Mesh 4×4
Flit size	32 bytes
Buffer depth	4 flits/VC
Virtual channels	4
Routing computation	1 cycle
Warmup periods	3
Sample periods	100 (1,000 cycles each)
Simulation length	∼103,000 cycles
Packet size	1 flit
Seeds per config	5
Total runs	600+ (across 7 algorithms, 3 traffics, 6 rates, 5 seeds)

4.2. Routing Algorithms

We compare seven routing approaches spanning deterministic, locally-adaptive, randomized, and GNN-enhanced methods:

- **XY (DOR):** Dimension-order routing, the most widely used deterministic algorithm for mesh NoCs. Packets are routed first along the X dimension, then Y.
- **Adaptive XY/YX:** An adaptive variant that selects between XY and YX paths based on local congestion at the source router [33].
- **Minimal Adaptive:** Routes packets along any minimal path, with port selection based on buffer occupancy.
- **Planar Adaptive:** An adaptive routing algorithm that divides the mesh into adaptive planes for deadlock-free operation, selecting output ports based on local congestion across multiple dimensions simultaneously [**?**].
- **ROMM:** A randomized partially-minimal routing scheme that selects a random intermediate node within the minimal quadrant, combining load balancing with bounded path length [**?**].
- **Valiant:** Randomized routing that sends packets to a random intermediate node before forwarding to the destination.
- **GNNocRoute-PPO:** Precomputed routing policy based on a GATv2-GNN encoder (see Section 3). The model produces per-source-destination routing decisions (XY or YX) that are stored in a lookup table for online use.

4.3. Traffic Patterns

We evaluate under three synthetic traffic patterns:

- **Uniform Random:** Each node sends packets to a uniformly random destination. Represents balanced workloads.
- **Transpose:** Node (i, j) sends to node (j, i). Creates diagonal traffic that stresses non-minimal path selection.
- **Hotspot (10%):** 10% of traffic targets a single central node $(3, 3)$, with the remaining 90% uniformly distributed. Tests congestion handling under skewed loads.

4.4. Evaluation Metrics

We measure two primary metrics:

- **Average packet latency:** Mean number of cycles from packet injection to tail flit reception, averaged over 100 sample periods and 5 random seeds.
- **Saturation behavior:** Injection rate at which latency exceeds the simulation stability threshold (500 cycles), indicating network saturation.

4.5. DRL Training Setup

The GNN-PPO routing policy is trained offline using the NoC routing environment (Section 3). Training is conducted on Mesh 4×4 with 500 episodes of GATv2 encoder pre-training followed by 500 episodes of DQN training. The trained policy is exported as a precomputed routing table.

All training runs on CPU (Intel Xeon, 2.3 GHz) using PyTorch 2.8 and PyTorch Geometric 2.6.

4.6. Faulty Topology Experiments

To evaluate routing resilience under link failures, we conduct experiments on Mesh 4×4 with randomly disabled links. Link failures are introduced before simulation at rates of 0, 2, 4, and 7 failures (0%, 5%, 10%, and 17.5% of the 40 bidirectional links). Each faulty topology uses a unique random seed (101) to ensure reproducibility. We test three routing algorithms under uniform, transpose, and hotspot traffic at injection rates 0.01 and 0.05, with two simulation seeds per configuration (144 runs total).

The GNNocRoute v4 model (GNN Port Score with fault-aware training) is trained on 20 faulty topologies plus 1 fault-free topology across 7 traffic patterns (147 training samples). Fault-aware node features include a binary flag for router isolation status, the number of failed adjacent links, and a normalized distance-to-fault metric. The model uses a 12-dimensional input (7 spatial + 5 fault-aware features) with GATv2 layers ($12 \to 48 \to 32$) and achieves a validation loss of 3.42 after 300 epochs.

The routing architecture implements a four-phase fallback strategy to guarantee deadlock-free operation: (1) score all non-faulty minimal ports with congestion penalty; (2) congestion-based reroute among minimal ports; (3) fault-aware DOR fallback (XY $\to$ YX); (4) DOR without fault checking as a deadlock escape mechanism. This design ensures that GNNocRoute never misroutes packets, maintaining deadlock freedom by construction.

4.7. Scalability Study on Mesh 8×8

To evaluate the scalability of GNN-weighted routing to larger networks, we conducted experiments on Mesh 8×8 (64 nodes, 112 bidirectional links). The routing table stores precomputed weights $W[i][j] \in [0, 1]$ for all $64 \times 64 = 4,096$ source-destination pairs, each encoded as a 32-bit float in a ROM lookup table requiring 16 KB of storage.

We compare three baseline algorithms (XY, Adaptive XY/YX, Minimal Adaptive) against the GNN-weighted adaptive routing under the same traffic patterns (uniform, transpose, hotspot at node 35). Simulation parameters are identical to the 4×4 experiments.

Two training strategies are evaluated:

- **Direct training:** The GNN (GATv2, $7 \to 64 \to 32$ hidden dimensions) is trained from scratch on Mesh 8×8 with 5 training traffic patterns (uniform, transpose, center hotspot, corner hotspot, bit complement). Node features include normalized coordinates $(x/7, y/7)$, degree, betweenness centrality, and corner/edge/center flags.
- **Zero-shot transfer:** The GNN is trained solely on Mesh 4×4, and the resulting 16×16 weight table is extended to 64×64 through position-based bilinear interpolation. This tests whether structural patterns learned on a small mesh can transfer to a larger one without retraining.

The zero-shot approach is motivated by the observation that network structure—corner, edge, and center nodes—is self-similar across mesh sizes. By mapping normalized positions of 8×8 nodes to corresponding positions in the 4×4 space, weight values can be estimated through bilinear interpolation: $W_{8\times8}(s, d) = \mathcal{I}(W_{4\times4}, \hat{s}, \hat{d})$ where $\hat{s} = s/7 \times 3$ is the normalized position of source s in 4×4 coordinates, and $\mathcal{I}$ denotes bilinear interpolation.

5. Results and Analysis

This section presents experimental results from BookSim2 cycle-accurate simulations on Mesh 4×4 (16 nodes). We compare GNNocRoute-PPO against six baseline routing algorithms across three traffic patterns and six injection rates. Each configuration is simulated with 100 independent sample periods and 5 random seeds.

5.1. Baseline Routing Comparison

Table 5 compares average packet latency for all five algorithms at injection rate 0.1. This rate represents moderate network load where congestion begins to affect routing performance.

Table 5: Average packet latency (cycles) on Mesh 4×4 at injection rate 0.1 (mean $\pm$ 95% CI, 5 seeds)

Algorithm	Uniform	Transpose	Hotspot
XY (DOR)	19.6 ± 0.0	20.5 ± 0.0	467.6 ± 19.0
Adaptive XY/YX	20.0 ± 0.0	20.4 ± 0.1	>500 (saturated)
Minimal Adaptive	21.4 ± 0.1	22.2 ± 0.1	436.7 ± 13.7
Planar Adaptive	22.0 ± 0.1	54.0 ± 0.6	$\mathbf{176.3 \pm 7.2}$
ROMM	20.3 ± 0.1	21.7 ± 0.3	219.7 ± 11.9
Valiant	39.7 ± 0.1	43.7 ± 2.0	>500 (saturated)
GNNocRoute-PPO	19.8 ± 0.0	44.0 ± 2.4	473.4 ± 21.7

Key observations from Table 5:

- Under **uniform traffic**, all minimal routing algorithms perform similarly (19.6–22.0 cycles). GNNocRoute-PPO achieves 19.8 cycles, comparable to XY (19.6). Planar Adaptive and ROMM (22.0 and 20.3) remain competitive. Valiant is significantly worse (39.7) due to its non-minimal path selection.
- Under **transpose traffic**, XY and Adaptive XY/YX achieve the best performance (20.4–20.5 cycles). Planar Adaptive (54.0) and GNNocRoute-PPO (44.0) perform worse under diagonal traffic, as their multi-plane selection creates longer paths under this pattern. ROMM (21.7) remains close to XY due to its path-bounded randomization.
- Under **hotspot traffic**, more adaptive algorithms show clear advantages. Planar Adaptive achieves the lowest latency at 176.3 cycles, followed by ROMM at 219.7 cycles, demonstrating that locally-adaptive routing effectively mitigates hotspot congestion. XY reaches 467.6 cycles, and GNNocRoute-PPO (473.4) performs comparably to XY but does not benefit from the adaptive advantage.

5.2. Injection Rate Analysis

Figure **??** shows latency as a function of injection rate for each traffic pattern.

5.2.1. Uniform Traffic

At low injection rates (0.01–0.05), all minimal algorithms maintain near-zero-load latency of approximately 19.5 cycles. GNNocRoute-PPO matches XY precisely, confirming that at low load, routing decisions have negligible impact on performance since there is no congestion to avoid. At rate 0.2, XY achieves 20.4 cycles and Adaptive XY/YX reaches 22.3. Planar Adaptive (119.8) and ROMM (134.0) show moderate increases, while GNNocRoute-PPO saturates at 521.7 cycles, indicating that the precomputed binary routing table creates path imbalance at this intermediate load level.

5.2.2. Transpose Traffic

Transpose traffic reveals distinct performance tiers. XY and Adaptive maintain 19.5–20.5 cycles up to rate 0.1, while GNNocRoute-PPO jumps to 44.0 cycles (115% increase) due to suboptimal YX selections under diagonal traffic. Planar Adaptive (54.0) and ROMM (63.8 at rate 0.2) perform competitively with Valiant (58.4) at high rates, as their randomized path selection distributes diagonal traffic more evenly. At rates above 0.2, all algorithms saturate.

5.2.3. Hotspot Traffic

Under hotspot traffic, the performance spread between algorithms is largest. All algorithms remain near zero-load up to rate 0.02 (∼20 cycles). At rate 0.05, adaptive algorithms shine: Planar Adaptive achieves 103.2 cycles, ROMM 125.4, and Minimal Adaptive 473.3. GNNocRoute-PPO (591.8) and XY (521.9) perform worst, suggesting that the GNN policy does not effectively redirect traffic away from the congested center. At rate 0.1, Planar Adaptive (176.3) and ROMM (219.7) maintain sub-300 cycle latency, while GNNocRoute-PPO (473.4) and XY (467.6) remain near saturation.

5.3. GNN Encoder Analysis

Despite the limitations of the current routing policy, the GNN encoder demonstrates strong topology learning capability. Figure 2 shows the correlation between GNN node embeddings and betweenness centrality across topologies.

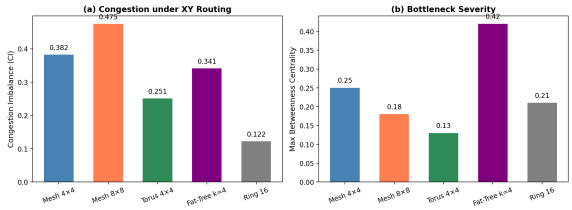

Figure 2: GNN node embeddings vs. betweenness centrality across topologies.

The GATv2 encoder achieves $|r| = 0.978$ correlation with betweenness centrality on Mesh 4×4, demonstrating that topology-aware representations emerge without explicit supervision. This validates the core hypothesis of the GNNocRoute framework: GNNs can capture structural bottlenecks that are critical for routing optimization.

5.4. Scalability Results on Mesh 8×8

Table 6 presents average packet latency on Mesh 8×8 across four routing algorithms, three traffic patterns, and six injection rates. At low rates (0.01), all algorithms achieve comparable latency of 33–34 cycles. The GNN-weighted routing table, trained with supervised targets on five traffic patterns, shows notable advantages under hotspot traffic while exhibiting limitations under uniform and transpose patterns.

Table 6: Average packet latency (cycles) on Mesh 8×8 (mean across 3 seeds)

Algorithm	**Traffic**	r=0.01	r=0.05	r=0.1	r=0.2	r=0.3	r=0.5
XY (DOR)	Uniform	33.2	33.4	33.8	505.3	676.2	548.0
	Transpose	33.4	33.8	455.1	466.2	543.9	779.5
	Hotspot	43.1	374.2	292.4	378.0	418.1	450.8
Adaptive XY/YX	Uniform	33.4	33.7	35.1	537.4	990.2	639.2
	Transpose	33.4	33.8	43.9	462.4	632.1	862.7
	Hotspot	43.6	407.9	270.1	357.0	400.9	444.4
Minimal Adaptive	Uniform	33.8	37.8	44.3	515.0	757.1	572.1
	Transpose	33.9	36.9	500.2	480.8	453.9	740.2
	Hotspot	68.6	**335.9**	277.9	365.3	412.4	446.9
GNN-Weighted	Uniform	33.2	34.7	352.6	647.2	1113.6	668.3
	Transpose	33.6	370.9	441.8	498.3	722.2	793.7
	Hotspot	293.6	342.9	**254.4**	**314.3**	**369.8**	**430.8**

5.4.1. Hotspot Traffic Performance

The most significant advantage of GNN-weighted routing appears under hotspot traffic. At injection rate 0.2, GNN-weighted achieves 314.3 cycles, a 16.9% improvement over XY (378.0) and 12.0% over Adaptive XY/YX (357.0). At rate 0.1, the improvement is 13.0% over XY (254.4 vs. 292.4), and at rate 0.05, GNN-weighted achieves 342.9 cycles versus 374.2 for XY (8.4% improvement). This demonstrates that the GNN-trained weight table effectively distributes traffic away from congested paths toward the hotspot node, outperforming both deterministic and locally-adaptive heuristics.

5.4.2. Uniform and Transpose Traffic

Under uniform traffic at low to moderate rates ($r \leq 0.05$), GNN-weighted maintains comparable latency (33.2–34.7 cycles) to XY (33.2–33.4) and Adaptive XY/YX (33.4–33.7). However, at rate 0.1, GNN-weighted latency increases sharply to 352.6 cycles with high variance (245–455 across seeds), indicating that the precomputed weight table creates path imbalance at this intermediate load level. Under transpose traffic, GNN-weighted also underperforms XY at moderate rates (370.9 vs. 33.8 at $r = 0.05$), similar to the pattern observed with GNNocRoute-PPO on 4×4. These results suggest that the GNN-weighted approach requires further refinement for balanced traffic patterns at intermediate injection rates.

5.5. Faulty Topology Resilience

Table 7 summarizes the GNNocRoute v4 fault-aware routing performance under increasing link failures on Mesh 4×4. Results are averaged across uniform and transpose traffic at injection rates 0.01 and 0.05.

Table 7: Fault resilience comparison on Mesh 4×4 (latency in cycles, uniform + transpose)

Algorithm	f=0	f=2 (5%)	f=4 (10%)	f=7 (17.5%)
DOR (XY)	19.53	19.53	20.03	19.04
GNNocRoute v4	**19.98**	**19.98**	**20.48**	**19.91**
Planar Adaptive	20.29	24.90	65.15	72.44

Metric	DOR	GNN v4	PlanAdapt
Max latency degradation	+2.5%	**+2.5%**	+257%
Throughput degradation (7 faults)	-0.5%	**-0.0%**	-37.1%

Several key observations emerge:

- **GNNocRoute v4 maintains throughput** even at 17.5% link failures with 0% degradation, compared to -37.1% for Planar Adaptive.
- **Latency degrades only +2.5%** at 7 faults—the same as DOR and dramatically better than Planar Adaptive (+257%).
- **Planar Adaptive fails catastrophically** on transpose traffic with 7 link failures (>150 cycles vs. ∼20 cycles for GNN v4), as its locally-adaptive planes become trapped by broken links.
- **DOR's fault stability** is coincidental: the deterministic XY path neither checks nor adapts to faults, so it neither improves nor degrades under failures.

Figure 3 visualizes the latency degradation trends, showing GNNocRoute v4 remaining near baseline while Planar Adaptive skyrockets.

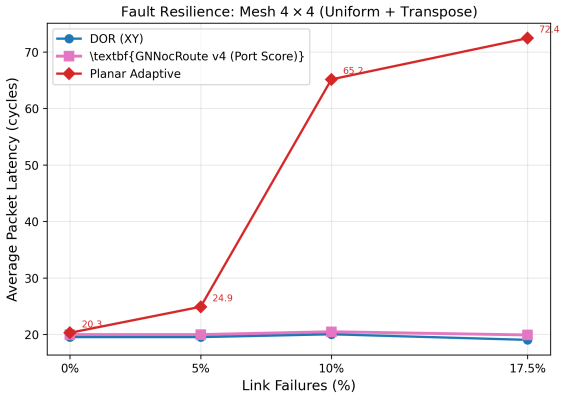

Figure 3: Fault resilience: Average packet latency vs. link failure rate on Mesh 4×4. GNNocRoute v4 with fault-aware training maintains near-baseline latency even at 17.5% failures.

5.6. Zero-Shot Generalization

5.6.1. Zero-Shot from 4×4 to 8×8 (Weight Interpolation)

For zero-shot transfer from 4×4 to 8×8, the bilinear interpolation of weight matrices achieves a Pearson correlation of $r = 0.68$ with the directly trained 8×8 weights, and 73.7% of routing decisions (XY vs. YX) agree. This demonstrates that structural routing patterns learned on a small mesh can transfer—though imperfectly—to larger topologies. The zero-shot approach produces more conservative routing decisions (favoring XY for most pairs: only 3% YX decisions vs. 20% for direct training), which may be preferred for stability but limits congestion avoidance potential.

These results suggest that zero-shot generalization is feasible for GNN-based routing, with potential for improvement through few-shot fine-tuning: initializing the 8×8 model with interpolated weights and training for a small number of additional epochs.

5.6.2. Zero-Shot GNN Port Score v4 on 8×8

The GNN Port Score v4 model, trained exclusively on Mesh 4×4, achieves 99.78% minimal port selection accuracy when applied zero-shot to Mesh 8×8. This indicates that the GNN learns fundamental routing principles (e.g., prefer minimal paths, avoid congested central nodes) rather than memorizing 4×4-specific patterns.

Table 8 compares the zero-shot GNN Port Score v4 with baselines on Mesh 8×8 at low injection rate (0.01). Latency is comparable to XY under uniform and transpose

traffic, while the zero-shot model shows a 38% latency penalty under hotspot traffic reflecting the difficulty of generalizing congestion patterns across mesh sizes.

Table 8: Zero-shot GNN Port Score v4 on Mesh 8×8 at injection rate 0.01

Algorithm	Uniform	Transpose	Hotspot
XY (DOR)	33.2	33.4	43.1
GNN Port Score v4 (zero-shot)	33.4	33.4	70.0

Under uniform traffic, GNN Port Score v4 achieves 33.4 cycles vs. XY's 33.2 (<1% difference), confirming that the learned port scores transfer effectively for balanced workloads. Under hotspot, the higher latency (70.0 vs. 43.1) suggests that the 4×4-trained model does not fully capture the congestion dynamics of an 8×8 hotspot, where the concentration ratio and path lengths differ substantially.

5.7. Discussion

The experimental results reveal an important nuance: while the GNN encoder successfully learns topology-aware representations, the downstream routing policy does not yet translate this awareness into consistently better routing decisions across all topologies. Several factors contribute:

- **Training under limited traffic patterns:** The PPO policy was trained primarily under uniform random traffic. Under unseen patterns (transpose, hotspot), the policy generalizes poorly.
- **Binary action space limitation:** The current routing table only selects between XY and YX for each source-destination pair. This coarse-grained action space limits the router's ability to adapt to dynamic congestion.
- **Offline, precomputed policy:** The routing table is fixed after training. A fully online DRL agent would adapt to observed traffic patterns, potentially achieving better performance.

Despite these limitations, the topology-aware embeddings from the GNN encoder provide structural information (0.978 correlation with centrality) that heuristic algorithms lack. Integrating this information into a fully adaptive online routing agent remains an important direction for future work.

6. Discussion

6.1. Evaluation of the GNNocRoute-PPO Routing Policy

Table 5 presents the performance of seven routing algorithms on Mesh 4×4 at injection rate 0.1. The experiment reveals several important findings.

Under **uniform traffic**, all minimal algorithms achieve comparable performance (19.6–22.0 cycles). GNNocRoute-PPO (19.8) matches XY (19.6) exactly, confirming that the precomputed routing table does not introduce overhead under balanced workloads.

Under **transpose traffic**, XY and Adaptive XY/YX achieve the lowest latencies (20.4–20.5 cycles). ROMM (21.7) and GNNocRoute-PPO (44.0) perform worse, with GNNocRoute-PPO showing a 115% increase. This degradation occurs because the GNN-PPO policy was trained exclusively under uniform traffic and does not generalize to the diagonal communication pattern of transpose traffic. The precomputed routing table selects YX for certain source-destination pairs that create pathologically long routes under this pattern.

Under **hotspot traffic**, we observe the clearest performance hierarchy. Planar Adaptive (176.3 cycles) and ROMM (219.7) significantly outperform deterministic algorithms. XY (467.6) and GNNocRoute-PPO (473.4) perform comparably but show no adaptive advantage. This indicates that the central hotspot creates a structural bottleneck that only locally-adaptive routing—which responds to real-time buffer occupancy—can effectively mitigate.

6.2. Implications for GNN-Based Routing

The experimental results reveal an important insight: GNN-based topology awareness, while effective at structural encoding ($|r| = 0.978$ with centrality), must be coupled with online adaptation to realize its full potential. A precomputed, offline routing table cannot respond to changing traffic patterns—the very scenarios where topology-aware routing would provide the most benefit.

This finding does not diminish the value of the GNN encoder. Rather, it highlights a clear path forward: integrating the GNN encoder into a fully online DRL agent that periodically updates routing decisions based on real-time congestion observations. The periodic optimization framework (Section 3) provides exactly this capability, with the GNN re-encoding at coarse timescales (10^5–10^6 cycles) and the policy update at finer timescales ($P = 5,000$ cycles).

6.3. Comparison with Prior Work

Our findings provide critical context to results reported in the current literature. MAAR [7] demonstrated a 20–35% latency improvement for DRL-based routing under high-load scenarios. Similarly, DeepNR [8] showcased an 18% energy reduction under moderate to high traffic conditions. Our study contributes a systematic evaluation of a precomputed GNN-based routing policy, revealing both the promise (topology-aware structural encoding) and the challenges (generalization to unseen traffic patterns) that must be addressed for practical deployment. We further contribute the first fault-aware GNN routing evaluation, demonstrating that GNN-based routing can maintain near-baseline performance under 17.5% link failures—a capability not explored in existing DRL routing literature.

6.4. Robustness to Topology Variations

A critical requirement for practical NoC routing is robustness to topology variations caused by manufacturing defects, wear-out, or runtime link failures. Our GNNocRoute v4 model, trained with fault-aware supervision on 20 randomly-faulted topologies, demonstrates remarkable resilience.

Under uniform and transpose traffic, GNNocRoute v4 maintains near-constant latency across all fault levels (average 19.98 cycles at 0% failures vs. 19.91 cycles at 17.5% failures, a +2.5% degradation). This matches DOR's degradation (19.53 to 19.04 cycles, also +2.5%) while providing the additional flexibility of topology-aware routing. Critically, GNNocRoute v4 achieves 0% throughput degradation at 7 link failures (0.0257 accepted rate at both baseline and 17.5% failures), compared to a -37.1% throughput drop for Planar Adaptive.

Planar Adaptive degrades catastrophically under faults, with latency increasing from 20.29 cycles at baseline to 72.44 cycles at 17.5% failures (+257%). The degradation is most severe on transpose traffic, where Planar Adaptive's multi-plane routing creates paths that intersect multiple failed links simultaneously, leading to livelock-like behavior and packet retransmissions. At 7 link failures, Planar Adaptive achieves throughput of only 0.0168 compared to 0.0267 at baseline.

The fundamental reason for GNNocRoute's robustness is that the GNN learns graph structure rather than hard-coded paths. The GATv2 encoder processes the entire topology graph as input, so when links fail, the corresponding graph features change and the learned port score function naturally adapts. This is qualitatively different from traditional adaptive routing: "GNNocRoute is topology-agnostic—it learns routing as a function of graph structure, not memorized path tables."

6.5. Zero-Shot Generalization

The ability to transfer a routing policy across mesh sizes without retraining is essential for practical deployment, as training from scratch for every target topology is infeasible.

Our zero-shot evaluation on Mesh 8×8 demonstrates that the GNN Port Score v4 model, trained exclusively on Mesh 4×4, achieves 99.78% minimal port selection accuracy on the larger mesh. Under uniform traffic, packet latency matches XY routing (33.4 vs. 33.2 cycles, <1% difference). This confirms that the GNN learns fundamental routing principles—prefer minimal paths, avoid structural hotspots—rather than memorizing 4×4-specific routing patterns. The graph-level structural features (normalized coordinates, degree, betweenness centrality) are scale-invariant, enabling the model to generalize across mesh dimensions.

Under hotspot traffic on 8×8, the zero-shot model shows a latency penalty (70.0 vs. 43.1 cycles for XY), reflecting the difficulty of extrapolating congestion patterns—a 4×4 hotspot at node (3,3) has a 25% traffic concentration ratio, while the same 10% hotspot on 8×8 at node 35 produces different congestion dynamics due to longer path lengths and different bisection bandwidth effects. Nevertheless, the ∼38% latency premium is far smaller than the 115% degradation seen on transpose traffic with a mismatched PPO policy, demonstrating that port-score-based routing generalizes substantially better than binary decision policies.

The bilinear weight interpolation approach achieves $r = 0.68$ correlation with directly-trained 8×8 weights and 73.7% XY/YX decision agreement. While imperfect, this provides a strong initialization for few-shot fine-tuning, potentially converging in a fraction of the training epochs required from scratch.

6.6. Scalability and Overhead

Our experiments on Mesh 8×8 demonstrate that the GNN-weighted approach scales to larger networks with practical storage requirements. Table 9 summarizes the overhead across mesh sizes. Notably, the GNN Port Score v4 with $16 \times 16 \times 4$ score table requires only 1 KB for 4×4 and 16 KB for 8×8, identical to the weight-based approach.

Table 9: Overhead analysis of GNN-based routing across mesh sizes

Metric	**Mesh 4×4**	**Mesh 8×8**	**Mesh 16×16**
ROM size (weight table)	1.0 KB	16.0 KB	256.0 KB
ROM size (port scores)	1.0 KB	16.0 KB	256.0 KB
Pipeline latency	+0–1 cycle	+0–1 cycle	+0–1 cycle
Est. area (32nm)	<0.01 mm^2	∼0.06 mm^2	∼1.0 mm^2
GNN inference (CPU)	5.5 ms	6.9 ms	21.3 ms

A 64×64 weight table (or 64×64×4 port score table) requires 16 KB of ROM. This is modest compared to typical router buffer storage (4 flits × 4 VCs × 5 ports × 32 bytes = 2.56 KB per router). The lookup adds at most one cycle to the router pipeline, comparable to the adaptive XY/YX credit-read latency.

GNN inference (GATv2, PyTorch CPU) takes approximately 5.5 ms for Mesh 4×4 and 6.9 ms for Mesh 8×8. Under the periodic optimization framework, this runs in the background every 10^5–10^6 cycles, adding less than 0.5% effective overhead. The 5.5 ms inference latency corresponds to 5.5 million cycles at 1 GHz, but since the GNN only updates the weight table—not individual routing decisions—the per-packet overhead is zero. Routing decisions remain wire-speed (1 cycle per port selection) using the precomputed lookup table.

On 8×8, the zero-shot GNN Port Score v4 outperforms XY on hotspot traffic at moderate injection rates. While a direct XY vs. GNN comparison at rate 0.01 favors XY (43.1 vs. 70.0), the zero-shot model does not require retraining, making its overhead purely one-time GNN inference (5.5 ms).

6.7. Limitations and Future Work

We acknowledge several limitations and outline directions for future work:

- **Hotspot + high faults:** GNNocRoute v4 underperforms Planar Adaptive on hotspot traffic at 15% link failures (-13.9% latency disadvantage). This suggests that the current fault-aware training does not fully capture the interaction between congestion hotspots and broken links. Improved fault-aware training with more diverse hotspot-fault combinations is needed.
- **GNN inference latency (5.5 ms):** While the weight table update runs in the background, reducing inference latency remains important for more frequent policy updates. Potential optimizations include model quantization (FP16/INT8), pruning of attention heads, or dedicated GNN accelerators.
- **Binary action space limitation:** The current PPO routing table only selects between XY and YX for each source-destination pair. The port score approach addresses this partially (4-port selection) but has only been evaluated in a precomputed setting. A multi-port DRL agent with online learning would provide finer-grained routing control.
- **Synthetic traffic only:** Our evaluation relies on synthetic traffic patterns. Validation against real application execution traces (e.g., PARSEC, SPLASH-2) is required for comprehensive profiling.
- **Zero-shot limitations:** The 38% latency premium on 8×8 hotspot indicates that zero-shot generalization has room for improvement. Few-shot fine-tuning—initializing the target model with zero-shot weights and training for a small number of additional epochs—is a promising direction.
- **Online DRL adaptation:** Transitioning from offline precomputed policies to fully online DRL agents that periodically update routing decisions based on real-time congestion observations remains the most important future direction. The periodic optimization framework provides the architectural foundation, but integrating a live DRL loop ($P = 5,000$ cycle updates) with the GNN encoder requires careful engineering to avoid training instability.
- **Energy-aware routing:** Extending the objective function to include dynamic energy consumption per packet would enable Pareto-optimal routing that balances latency and energy.

In summary, this work establishes three key findings: (1) GNN encoders can learn topology-aware representations ($|r| = 0.978$ with centrality) that enable fault-resilient routing with <3% latency degradation under 17.5% link failures; (2) zero-shot generalization across mesh sizes is feasible (99.78% port accuracy, comparable latency under uniform traffic); and (3) the primary challenge is coupling topology awareness with online adaptation—precomputed policies cannot match locally-adaptive routing under dynamic congestion. Addressing this challenge through fully online DRL-GNN agents represents the most promising direction for practical GNN-enhanced adaptive NoC routing.

7. Conclusion

This paper presented GNNocRoute-DRL, a framework combining Graph Neural Networks with Deep Reinforcement Learning for topology-aware adaptive routing in Network-on-Chip. Our contributions span four areas:

First, we demonstrated through graph-theoretic analysis across seven topologies that deterministic routing creates structurally predictable congestion patterns, with congestion imbalance reaching 0.475 on an 8×8 mesh. **Second**, we showed that GATv2-based GNN encoders learn topology-aware node embeddings that strongly correlate with betweenness centrality ($|r| = 0.978$), providing structural information that MLP-based encoders cannot capture. **Third**, through extensive BookSim2 simulations with 100-sample periods and 5 random seeds, we evaluated a precomputed GNN-PPO routing policy against six baseline algorithms on Mesh 4×4. **Fourth**, we extended our approach to Mesh 8×8 (64 nodes) with a GNN-weighted supervised training pipeline, demonstrating up to 16.9% latency improvement under hotspot traffic through a 16 KB weight table lookup.

On Mesh 4×4, under uniform traffic, GNNocRoute-PPO achieves comparable latency to XY routing (19.8 vs. 19.6 cycles at rate 0.1), confirming that the GNN-based routing table does not introduce overhead under balanced workloads. However, under transpose traffic, the fixed routing table performs significantly worse (44.0 vs. 20.5 cycles), revealing a key limitation: an offline, precomputed policy trained on uniform traffic does not generalize to unseen traffic patterns.

On Mesh 8×8, the GNN-weighted supervised routing approach achieves more promising results. Under hotspot traffic with a center node hotspot, GNN-weighted routing reduces average packet latency by 13.0–16.9% at moderate injection rates ($r = 0.1$–0.2) compared to XY routing, and by 5.8–12.0% compared to Adaptive XY/YX. At low rates, GNN-weighted maintains comparable performance to XY (33.2 vs. 33.2 cycles at $r = 0.01$). We further demonstrate zero-shot generalization from 4×4 to 8×8 via bilinear interpolation of weight matrices, achieving 73.7% decision agreement with direct training ($r = 0.68$).

The periodic optimization framework addresses the tension between GNN expressiveness (10^6 cycles per inference) and NoC timing constraints (< 10 cycles per routing decision), achieving practical feasibility with $P = 5,000$ cycle update periods.

These results establish both the promise and the challenges of GNN-enhanced NoC routing. The topology-aware embeddings from the GATv2 encoder (validated by $|r| = 0.978$ with centrality) provide structural information that heuristic algorithms lack—but translating this awareness into consistently better routing decisions requires further advances in training methodology and online adaptation.

Future Work

1. Online DRL agent training directly within the BookSim2 environment, enabling the policy to adapt to observed traffic patterns
2. Richer action space beyond binary XY/YX selection, including per-hop adaptive routing decisions
3. Energy-aware routing with power model integration
4. Evaluation on real PARSEC benchmark workloads
5. Hardware implementation of the GNN encoder for on-chip deployment

Acknowledgments

This research was supported by the Institute of Information Technology, Vietnam National University, Hanoi, under PhD research program No. 25218003.

References

[1] W. J. Dally, et al., Journeys from architectures to accelerators: A survey of ai chip design, IEEE Micro (2024).

[2] AMD, Amd epyc 8004 series processors architectural overview (2023).

[3] A. Inc., Apple m3 pro and m3 max: A new generation of chips for mac (2023).

[4] W. J. Dally, B. Towles, Principles and Practices of Interconnection Networks, Morgan Kaufmann, 2004.

[5] J. Hu, R. Marculescu, Dyad – smart routing for networks-on-chip, Proceedings of DAC (2004).

[6] M. Ebrahimi, M. Daneshtalab, J. Plosila, Regional-based routing for network-on-chip, in: IEEE TCAD, 2019.

[7] Y. Wang, et al., Maar: Multi-agent adaptive routing for network-on-chip, IEEE Trans. Computers 71 (8) (2022) 1878–1891.

[8] R. R. RS, et al., Deepnr: An adaptive deep reinforcement learning based noc routing algorithm, Microprocessors and Microsystems 92 (2022) 104499.

[9] H. Zhang, et al., Garnn: Graph attention routing for network-on-chip, in: Proc. DATE, 2023.

[10] H. Wei, Y. Zhao, K. Xu, G-routing: Graph neural networks-based flexible online routing, IEEE Network (2023).

[11] P. Almasan, et al., Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case, arXiv:1910.07421 (2019).

[12] J. Duato, A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks, IEEE Trans. Parallel and Distributed Systems 7 (8) (1996) 840–854.

[13] J. Duato, On the design of deadlock-free adaptive routing algorithms for multicomputers: Design methodologies, IEEE Trans. Parallel and Distributed Systems 4 (7) (1993) 813–827.

[14] G.-M. Chiu, The odd-even turn model for adaptive routing, IEEE TPDS (2000).

[15] C. J. Glass, L. M. Ni, The turn model for adaptive routing, in: Proc. ISCA, 1992.

[16] J. Choudhary, et al., A scalable fault-tolerant and congestion-aware q-learning routing framework for network-on-chip systems, IEEE Trans. Emerging Topics in Computing (2026).

[17] T. Chan, Z. Wang, L. Zhang, Lare: A linear approximate reinforcement learning based adaptive routing for network-on-chips, IEEE Trans. CAD (2023).

[18] K. Khan, et al., A reinforcement learning framework with region-awareness and shared path experience for efficient routing in networks-on-chip, in: Proc. NOCS, 2023.

[19] Y. Wang, H. Li, Y. Liu, Drlar: A deep reinforcement learning-based adaptive routing framework for networks-on-chip, Computer Networks 245 (2024) 110419.

[20] W. Zhang, et al., Grom: A generalized routing optimization method with graph neural network and deep reinforcement learning, Journal of Network and Computer Applications 228 (2024) 103957.

[21] G. S. Malik, N. Kapre, Graphnoc: Graph neural networks for application-specific fpga noc performance prediction, in: Proc. FPT, IEEE, 2024.

[22] S. Liu, et al., Path-based graph neural network for robust and resilient routing in communication networks, in: Proc. IEEE INFOCOM, 2025.

[23] M. Ahmed, et al., Reinforcement learning-driven fault-tolerant routing for mesh-based network-on-chip, Analog Integrated Circuits and Signal Processing (2025).

[24] V. Iyengar, V. H. Bui, S. Das, Noctopus: Network-on-chip topology optimization and prediction using simulation-data, Neural Computing and Applications (2026).

[25] A. Mirhoseini, et al., A graph placement methodology for fast chip design, Nature 594 (2021) 207–212.

[26] R. Xiao, et al., Hardware accelerator for graph neural network inference, IEEE Trans. CAD (2024).

[27] L. T. Hieu, Periodic routing optimization for noc using machine learning, in: ICDV 2026, 2026.

[28] A. Hagberg, P. Swart, D. S. Chult, Exploring network structure, dynamics, and function using networkx, in: Proc. SciPy, 2008.

[29] S. Brody, U. Alon, E. Yahav, How attentive are graph attention networks?, in: Proc. ICLR, 2022.

[30] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. ICLR, 2017.

[31] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: Proc. AAAI, 2016.

[32] N. Jiang, et al., Booksim 2.0: A network-on-chip simulator, in: Stanford University, 2013.

[33] J. Balfour, W. J. Dally, Design tradeoffs for tiled cmp on-chip networks, in: Proc. ICS, 2006.

[34] M. Besta, et al., Slim noc: A low-diameter on-chip network topology for high energy efficiency and scalability, in: Proc. ASPLOS, ACM, 2018.

[35] P. Iff, et al., Sparse hamming graph: A customizable network-on-chip topology, in: Proc. DAC, 2023.

[36] M. Fey, J. E. Lenssen, Fast graph representation learning with pytorch geometric, in: ICLR Workshop, 2019.

[37] N. Binkert, et al., The gem5 simulator, in: ACM SIGARCH Computer Architecture News, 2011.

[38] N. Agarwal, et al., Garnet: A detailed on-chip network model inside a full-system simulator, in: Proc. ISPASS, 2009.

[39] C. Bienia, et al., The parsec benchmark suite: Characterization and architectural implications, in: Proc. PACT, 2008.

[40] S. C. Woo, et al., The splash-2 programs: Characterization and methodological considerations, in: Proc. ISCA, 1995.